# DATABASE MANAGEMENT SYSTEM (CT 652)

*Tribhuvan University | Institute of Engineering | BCT | Year III / Part II*

## CHAPTER 9: Data Warehousing and Data Mining

Syllabus Hours: 6 hrs | Marks Weight: 12 marks

*Complete Exam-Ready Answers | All Past Questions Covered (2070-2082)*

### CHAPTER 9 SYLLABUS TOPICS

* **9.1** Decision Support Systems and Data Warehousing
* **9.2** Data Warehouse: Definition, Characteristics, and Architecture
* **9.3** OLTP vs OLAP
* **9.4** Multidimensional Data Model: Dimensions, Measures, and Data Cube
* **9.5** Schemas for Multidimensional Data: Star, Snowflake, Fact Constellation
* **9.6** ETL Process: Extract, Transform, Load
* **9.7** OLAP Operations: Roll-up, Drill-down, Slice, Dice, Pivot
* **9.8** Data Mining: Concepts and Tasks
* **9.9** Classification: Decision Trees and Naive Bayes Classifier
* **9.10** Clustering: K-Means Clustering
* **9.11** Association Rule Mining: Apriori Algorithm
* **9.12** Data Mining vs Machine Learning; Applications of Data Mining

## SECTION 1: Decision Support Systems and Data Warehousing

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh*

**Q1.** What is a Decision Support System (DSS)? What is a Data Warehouse? Explain the key characteristics of a Data Warehouse as defined by W.H. Inmon. Why cannot a transactional database (OLTP) be directly used for decision support?

**[2+2+4]**
*2081 Chaitra 2079 Chaitra 2075 Bhadra*

**Answer:** Decision Support Systems and Data Warehouse

**Decision Support System (DSS):** A Decision Support System is an information system that supports organizational decision-making activities. It analyzes business data from multiple sources to help managers, analysts, and executives make informed strategic decisions. Examples: sales trend analysis, financial forecasting, customer segmentation, supply chain optimization.

**Data Warehouse:** A data warehouse (DW) is a large, subject-oriented, integrated, non-volatile, and time-variant collection of data organized to support management decision making. Defined by W.H. Inmon (1990) as: "A subject-oriented, integrated, nonvolatile, and time-variant collection of data in support of management's decisions."

## The Four Key Characteristics of a Data Warehouse (Inmon's Definition):

| Characteristic | Definition | Example |
|---|---|---|
| Subject-Oriented | Organized around MAJOR SUBJECTS of the enterprise (customers, sales, products), rather than around specific applications (order entry, billing). Focuses on the things that matter for analysis. | Operational DB: organized around "Order Processing" application. DW: organized around "Sales" subject for analysis. |
| Integrated | Data from MULTIPLE HETEROGENEOUS sources (OLTP systems, flat files, ERP, CRM, external data) is CLEANED, STANDARDIZED, and integrated into a consistent format. Resolves inconsistencies in naming, encoding, units. | One system stores gender as M/F, another as 1/0, another as Male/Female. DW standardizes all to Male/Female. |
| Non-Volatile | Data in the DW is NEVER modified or deleted once loaded. New data is ADDED but old data remains. The DW is a read-only historical archive - not updated transaction by transaction. | If a customer address changes, operational DB updates the record. DW keeps BOTH old AND new address with timestamps. |
| Time-Variant | Data in the DW contains a TIME DIMENSION - every record is associated with a specific time period. Enables historical analysis and trend discovery. Data is stored for 5-10+ years. | Operational DB: current salary only (50,000). DW: salary history - 2022:40000, 2023:45000, 2024:50000. |

## Why OLTP Database Cannot Be Directly Used for Decision Support:

| Problem | OLTP Database Behavior | Impact on Decision Support |
|---|---|---|
| Performance Conflict | OLTP optimized for fast INSERT/UPDATE/DELETE for concurrent transactions. | Complex analytical queries (aggregations, joins across years) would block and slow down OLTP transactions. |
| Query Type Mismatch | OLTP handles short, simple queries on small sets of rows. | DSS needs complex queries scanning millions of rows: "Total sales by region for last 5 years". |
| Historical Data Absent | OLTP only stores CURRENT state. Old records overwritten or deleted. | Cannot analyze trends: "How have sales grown over the past 10 years?" |
| Data Fragmentation | Data scattered across multiple operational systems (ERP, CRM, billing). | Cannot get a unified view across all systems without complex joins. |
| No Subject Orientation | OLTP organized by application function, not by analysis subject. | Difficult to query across all subjects (customer + product + geography). |
| Schema Complexity | Highly normalized (3NF/BCNF) for update efficiency. Many tables. | Complex analytical queries require many JOINs - slow and hard to write. |

**Q2.** Explain the architecture of a Data Warehouse. Describe the three-tier data warehouse architecture: data sources tier, data staging area, and data warehouse / data mart tier.

**[2+6]**
*2081 Ashwin 2079*
*Chaitra 2076 Baisakh*

**Answer:** Data Warehouse Architecture

**Three-Tier Data Warehouse Architecture**

```
TIER 1: DATA SOURCES                                (Heterogeneous sources)
+----------+  +----------+  +----------+  +-----------+
| OLTP DB  |  | ERP/CRM  |  | Flat Files|  | External  |
|(MySQL,   |  |(SAP,     |  |  (CSV,    |  | Data      |
| Oracle)  |  | Dynamics)|  |  Excel)   |  |(Web, API) |
+----------+  +----------+  +----------+  +-----------+
      |            |              |              |
      +------------+--------------+--------------+
                         | ETL PROCESS
                         v
TIER 2: DATA STAGING AREA / ETL LAYER
+-------------------------------------------------------+
|  EXTRACT (read from sources)                          |
|  TRANSFORM (clean, standardize, integrate, aggregate) |
|  LOAD (write to data warehouse)                       |
+-------------------------------------------------------+
                         |
                         v
TIER 3: DATA WAREHOUSE / PRESENTATION LAYER
+-------------------------------------------------------+
|  ENTERPRISE DATA WAREHOUSE (EDW)                      |
|  (Integrated, historical, subject-oriented data)      |
|  +-----------------+  +-----------------+           |
|  |  Data Mart:     |  |  Data Mart:     |           |
|  |  Sales          |  |  Finance        |           |
|  +-----------------+  +-----------------+           |
+-------------------------------------------------------+
                         |
             +-----------+-----------+
             |           |           |
      +------+--+   +-----+---+   +---+--------+
      | OLAP    |   | Reports |   | Data Mining|
      | Tools   |   | (SQL)   |   | Tools      |
      +---------+   +---------+   +------------+
      |  Users: Analysts, Managers, Executives              |
```

**Note:** The ETL layer is the most complex and time-consuming part of a data warehouse project (up to 70% of DW project effort). The staging area is a temporary workspace where data is cleaned before loading.

**Data Warehouse Architecture Components:**

| Component | Description | Role |
|---|---|---|
| Data Sources (Tier 1) | Heterogeneous operational systems: OLTP databases, ERP/CRM systems, flat files, external data feeds, APIs. | Provide raw data. Different formats, encodings, and schemas. |
| ETL (Staging) Layer (Tier 2) | Extract-Transform-Load process. The integration middleware between sources and the warehouse. | Extract data from sources, clean and transform it, load into DW. |
| Enterprise Data Warehouse (Tier 3) | The central, integrated repository of historical enterprise data. The single source of truth. | Stores clean, integrated, historical, subject-oriented data for analysis. |
| Data Marts | Smaller, department-specific subset of the DW focused on one business area (Sales DM, Finance DM, HR DM). | Faster queries for specific departments. Pre-aggregated for their needs. |
| Metadata Repository | Stores information about the DW itself: source-to-target mappings, transformation rules, data lineage, refresh schedules. | Documents the DW structure. Essential for DW management and debugging. |
| OLAP Server | Multidimensional analysis engine: MOLAP (stores cubes), ROLAP (relational cubes), HOLAP (hybrid). | Enables fast multidimensional queries (slice, dice, drill-down, roll-up). |
| Query/Reporting Tools | BI tools for end-user access: SQL queries, reports, dashboards, visualizations. | Power BI, Tableau, Oracle BI, Crystal Reports. User interface to DW. |

**Q3.** Compare OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing) in detail. How does a data warehouse support OLAP?

**[4+4]**
*2082 Kartik 2080*
*Ashwin 2078 Chaitra*

**Answer:** OLTP vs OLAP Comparison

| Feature | OLTP (Online Transaction Processing) | OLAP (Online Analytical Processing) |
|---|---|---|
| Full Form | Online Transaction Processing | Online Analytical Processing |
| Primary Purpose | Manage day-to-day business OPERATIONS: process orders, update records, handle transactions. | Support ANALYTICAL DECISIONS: analyze trends, patterns, historical data for strategic decisions. |

| | | |
|---|---|---|
| Users | Clerks, cashiers, tellers, order entry operators (many users simultaneously). | Analysts, managers, executives (fewer users, complex queries). |
| Operations | Simple, short INSERT, UPDATE, DELETE transactions. Frequent writes. | Complex SELECT queries with aggregations, joins, GROUP BY over large datasets. Read-only. |
| Query Complexity | Simple: SELECT * FROM Order WHERE order_id = 12345. | Complex: "Total sales by region, product, and quarter for last 3 years." |
| Data Volume | Current operational data. DB size: GB range. | Historical integrated data. DW size: TB to PB range. |
| Response Time | Milliseconds (fast transaction commit). | Seconds to minutes (complex aggregation over massive data). |
| Data Age | Current state only (latest values). | Historical data over years (5-10 years). |
| Schema Design | Highly normalized (3NF/BCNF) to minimize redundancy and support updates. | Denormalized (Star/Snowflake schema) to minimize joins and speed up reads. |
| Backup & Recovery | Critical - transactions must not be lost. | Periodic bulk reload from sources. Can be rebuilt from OLTP data. |
| Index Design | Many indexes on update-heavy tables for fast lookup. | Few indexes; uses bitmap indexes, materialized views, aggregation tables. |
| Example Systems | MySQL, Oracle OLTP, SQL Server, PostgreSQL (transactional use). | Amazon Redshift, Google BigQuery, Snowflake, Oracle OLAP, Microsoft SSAS. |
| Example Query | SELECT balance FROM Account WHERE acc_id = 1001; | SELECT region, SUM(amount) FROM Sales WHERE year=2023 GROUP BY region; |

**Most Repeated:** "Compare OLTP and OLAP with examples." This question appears in almost every examination. Key points: OLTP=operational/transactional/normalized/current data. OLAP=analytical/historical/denormalized/aggregated. Always give at least 6-8 comparison points.

## SECTION 2: Multidimensional Data Model and Warehouse Schemas

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh*

**Q4.** What is the Multidimensional Data Model? Define: Dimension, Measure, Dimension Hierarchy, and Data Cube. Illustrate with a Sales Data Cube example.

**[2+4+2]**

*2081 Chaitra 2079 Chaitra 2075 Bhadra*

**Answer:** Multidimensional Data Model

**Multidimensional Data Model:** The multidimensional data model organizes data as a multi-dimensional CUBE rather than flat tables. Data is viewed from multiple perspectives (dimensions). It is the core model of OLAP and data warehousing, allowing analysts to slice and dice data along any combination of dimensions.

**Key Concepts:**

| Concept | Definition | Example (Sales DW) |
|---|---|---|
| Dimension | A PERSPECTIVE or SUBJECT by which data is analyzed. Dimensions define the axes of the data cube. Each dimension has a set of attribute values. | Time (2021, 2022, 2023), Location (City, Region, Country), Product (Item, Category, Brand), Customer (Name, Segment) |
| Measure (Fact) | A NUMERICAL VALUE being analyzed - the thing you want to measure. Stored at the intersection of dimension values. | Total Sales Amount (Rs.), Units Sold, Profit, Discount Given, Transaction Count. |
| Dimension Hierarchy | Levels of aggregation within a dimension. Allows ROLL-UP (aggregate) and DRILL-DOWN (detail) operations. | Time: Day -> Month -> Quarter -> Year. Location: City -> District -> Province -> Country. |
| Data Cube | A multidimensional array of measure values indexed by dimension values. Can have 2, 3, or more dimensions (an n-dimensional hypercube). | A 3D cube indexed by (Time x Location x Product) with Sales Amount as the measure value at each cell. |
| Cell | A single data point in the cube - the intersection of specific values from all dimensions. | Sales in Kathmandu, of Laptop, in Q1-2023 = Rs. 500,000. |
| Sparsity | Most cells in a real data cube are EMPTY (no sales for every product in every city every day). Efficient storage handles sparse cubes. | Not every product sold in every city every day - most cells are zero/NULL. |

**3D Sales Data Cube**

```
                          TIME DIMENSION
                    Q1    Q2    Q3    Q4
                  +-----+-----+-----+-----+
       Laptop     | 500 | 480 | 520 | 600 |   PRODUCT
       Phone      | 300 | 320 | 290 | 400 |   DIMENSION
   K   Tablet     | 150 | 160 | 140 | 200 |
   A            +-----+-----+-----+-----+
   T   Laptop     | 400 | 420 | 390 | 450 |
   H   Phone      | 250 | 230 | 260 | 300 |  Numbers
   M   Tablet     | 120 | 110 | 130 | 150 |  = Sales
   A            +-----+-----+-----+-----+     Amount
```

```
N
D   Laptop      | 200 | 190 | 210 | 240 |
U   Phone       | 100 |  90 | 110 | 130 |
    Tablet      |  80 |  70 |  85 |  90 |
 LOCATION: Kathmandu / Pokhara / Lalitpur

Dimensions: Time (Q1-Q4), Location (cities), Product (items)
Measure:    Sales Amount (values in cells)
Cell [Laptop, Kathmandu, Q1] = 500 (thousand Rs.)
```

**Note:** A data cube can have many more than 3 dimensions: add Customer, Channel, Salesperson, etc. With n dimensions it becomes a hypercube. The OLAP engine lets you view any "slice" of the cube.

**Q5.** What is a Star Schema in data warehousing? Draw a Star Schema for a Sales Data Warehouse. How does it differ from a Snowflake Schema and a Fact Constellation (Galaxy Schema)? Compare all three.

**[2+4+4]**
*2082 Kartik 2080*
*Ashwin 2078 Baishakh*

**Answer:** Star Schema, Snowflake Schema and Fact Constellation

**1. Star Schema:**

**Star Schema:** In a Star Schema, there is ONE central FACT TABLE surrounded by multiple DIMENSION TABLES - like a star. The fact table contains FOREIGN KEYS to all dimension tables plus NUMERIC MEASURES. Dimension tables are denormalized (not in 3NF) for query performance.

**Star Schema - Sales Data Warehouse**

```
DIM_TIME                    DIM_LOCATION
+-----------+               +------------------+
| time_id PK|               | location_id PK   |
| day       |               | city             |
| month     |               | district         |
| quarter   |               | province         |
| year      |               | country          |
+-----------+               +------------------+
        \                         /
         \                       /
   +-----------------------------------+
   |       FACT_SALES  (center)        |
   |-----------------------------------|
   | time_id        FK -> DIM_TIME     |
   | location_id    FK -> DIM_LOCATION|
   | product_id     FK -> DIM_PRODUCT  |
   | customer_id    FK -> DIM_CUSTOMER|
```

```
                |-----------------------------------|
                | sales_amount    (MEASURE)         |
                | units_sold      (MEASURE)         |
                | discount        (MEASURE)         |
                +-----------------------------------+
                     /                     \
                    /                       \
            +-----------+              +------------------+
            | time_id PK|              | customer_id PK   |
            | DIM_PRODUCT              | customer_name    |
            | product_id PK            | city             |
            | product_name|            | segment          |
            | category  |              +------------------+
            | brand     |              DIM_CUSTOMER
            +-----------+
            DIM_PRODUCT
```

**Note:** Star Schema: fact table at center, denormalized dimension tables around it. Simple to query - usually only ONE JOIN level from fact to dimension. Fast for OLAP queries.

## 2. Snowflake Schema:

**Snowflake Schema:** In a Snowflake Schema, the dimension tables of a Star Schema are NORMALIZED into multiple related tables - creating a snowflake-like shape. For example, the Product dimension is split into Product -> Category -> Brand tables. This eliminates redundancy in dimension tables but requires more JOINs.

**Snowflake Schema - Normalized Dimension Tables**

```
     Star Schema Dimension:          Snowflake Schema Dimensions:

     DIM_PRODUCT (denormalized)       DIM_PRODUCT
     +-------------------+           +------------------+
     | product_id PK     |           | product_id PK    |
     | product_name      |           | product_name     |
     | category_name     |           | category_id FK --+---> DIM_CATEGORY
     | brand_name        |           +------------------+     +------------+
     +-------------------+                                    | category_id|
     (redundant category/brand                                | cat_name   |
      names repeated per product)                             | brand_id FK+-
-> DIM_BRAND
                                                             +------------+
+----------+

| brand_id |
```

```
| brand_nm |
+----------+

    Snowflake normalizes dimensions to avoid redundancy.
    Cost: more JOINs needed in queries. Benefit: less storage.
```

**Note:** Snowflake is more normalized than Star but requires more JOIN operations. Most commercial DW tools prefer Star Schema for query performance.

## 3. Fact Constellation (Galaxy Schema):

**Fact Constellation (Galaxy Schema):** A Fact Constellation (also called Galaxy Schema) contains MULTIPLE FACT TABLES sharing some dimension tables. Used when a DW needs to track multiple related business processes (e.g., both Sales and Purchases share the Product and Time dimensions).

**Fact Constellation - Multiple Fact Tables**

```
    DIM_TIME          DIM_PRODUCT          DIM_LOCATION
    +--------+       +----------+          +----------+
    |time_id |       |product_id|          |loc_id    |
    |...     |       |...       |          |...       |
    +--------+       +----------+          +----------+
        |          /    |     \                |
        |         /     |      \               |
   +----------+         |   +----------------+
   |FACT_SALES|         |   |FACT_PURCHASE   |
   |----------|         |   |----------------|
   |time_id   |         |   |time_id  FK     |
   |product_id|         |   |product_id FK   |
   |loc_id    |         |   |supplier_id FK  |
   |sales_amt |         |   |purchase_cost   |
   |units_sold|         |   |units_received  |
   +----------+         |   +----------------+
                         |
                   DIM_SUPPLIER (only for Purchase)
                   +------------------+
                   |supplier_id       |
                   |supplier_name ... |
                   +------------------+

  Both fact tables SHARE DIM_TIME and DIM_PRODUCT dimensions.
```

**Note:** Fact Constellation is the most flexible DW schema. Allows comparing sales vs purchases across the same time and product dimensions.

**Star vs Snowflake vs Fact Constellation - Comparison:**

| Feature | Star Schema | Snowflake Schema | Fact Constellation |
|---|---|---|---|
| Fact Tables | ONE fact table. | ONE fact table. | MULTIPLE fact tables. |
| Dimension Normalization | Denormalized (flat, all attributes in one table). | Normalized (split into related tables). | Either - mixed. |
| Query JOINs | Simple - one level JOIN. | Complex - multiple JOIN levels. | Multiple joins; cross-fact queries. |
| Storage | More redundancy in dimensions. | Less redundancy. | Most comprehensive. |
| Query Performance | FASTEST - fewer JOINs. | Slower than star. | Depends on query. |
| Maintenance | Easier - fewer tables. | More complex. | Most complex. |
| Best For | Most OLAP/BI scenarios; simple, fast. | Large dimension tables with redundancy concerns. | Enterprise DW with multiple business processes. |

## SECTION 3: ETL Process and OLAP Operations

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2070 Magh*

**Q6.** What is the ETL Process in data warehousing? Explain each step: Extract, Transform, and Load with examples. What are common data quality problems addressed during ETL?

**[2+4+2]**
*2081 Ashwin 2079 Chaitra 2077 Chaitra*

**Answer:** ETL - Extract, Transform, Load

**ETL (Extract, Transform, Load):** ETL is the process of moving data FROM multiple heterogeneous source systems (OLTP DBs, files, APIs) INTO the data warehouse. ETL is the backbone of data warehousing - it handles data integration, cleaning, standardization, and loading. ETL typically runs in batch mode at regular intervals (nightly, weekly).

**ETL Process - Three Stages:**

| Stage | What Happens | Key Activities | Example |
|---|---|---|---|
| EXTRACT | Read raw data from multiple source systems. Sources may be in different formats, locations, and technologies. | Connect to source DBs (JDBC/ODBC). Read flat files (CSV, Excel). Call APIs (REST/SOAP). Extract full loads OR incremental (only changed records). | Extract orders from MySQL OLTP DB. Extract HR data from SAP ERP. Extract market data from external CSV files. |
| TRANSFORM | Clean, standardize, integrate, aggregate, and restructure data. The most complex ETL stage. Applied in the staging area. | Data cleaning (remove nulls, fix errors). Standardization (M/F -> Male/Female). De-duplication (merge duplicate records). Derived columns (full_name = first + last). Key generation (surrogate keys). Business rules (categorize salary ranges). | Source: gender=1 or 0. Target: gender=Male or Female. Source: date=15/04/2023. Target: date=2023-04-15. Remove duplicate customer records. Compute total_price = qty * unit_price. |
| LOAD | Load transformed data into the data warehouse. Can be full load (replace all) or incremental load (append new/changed records only). | Initial full load (first time). Incremental/delta load (ongoing). Slowly Changing Dimension (SCD) handling. Index rebuild after load. Statistics update for query optimizer. | Nightly load: append new sales records from today. Weekly full refresh of product catalog. Update SCD Type 2 (new row for changed customer address). |

## Common Data Quality Problems Addressed During ETL:

| Data Quality Issue | Description | ETL Solution |
|---|---|---|
| Missing Values (NULL) | Required fields are NULL or empty. | Replace NULL with default value, infer from related data, or reject record. |
| Inconsistent Encoding | Same concept encoded differently across systems. | Standardize: M/F/Male/Female -> Male/Female. Yes/No/1/0 -> True/False. |
| Duplicate Records | Same entity appears multiple times in source data. | De-duplication using match keys (name + DOB + phone). Fuzzy matching. |
| Invalid Values | Values outside allowed ranges or of wrong type. | Validate: age=250 is invalid. Date=30/02 is invalid. Reject or flag. |
| Referential Integrity Violations | FK values that don't exist in referenced tables. | Look up valid values. Assign "unknown" dimension for unresolvable FKs. |
| Date/Format Inconsistencies | Different date formats (15/4/2023 vs 2023-04-15) across sources. | Standardize all dates to ISO 8601 (YYYY-MM-DD) format. |

| | | |
|---|---|---|
| Unit Inconsistencies | Same measure in different units across systems. | Standardize: convert all currencies to NPR, all weights to kg. |

**Q7.** What are OLAP Operations? Explain Roll-up, Drill-down, Slice, Dice, and Pivot operations with diagrams using a Sales data cube example.

**[2+8]**
*2082 Kartik 2081*
*Chaitra 2079 Chaitra*

**Answer:** OLAP Operations on the Data Cube

OLAP (Online Analytical Processing) provides a set of operations to explore and analyze multidimensional data in the data cube. These operations allow analysts to view data at different levels of detail and from different perspectives.

### Initial Data Cube for Examples:

**Sales Data Cube (Time x Location x Product)**

```
  Dimensions: Time (Q1,Q2,Q3,Q4), Location (KTM,PKR,BTW), Product
(Laptop,Phone)
  Measure: Sales Amount (Rs. thousands)

  Time: Q1
  Location  | Laptop | Phone |
  ----------+--------+-------|
  Kathmandu |   500  |  300  |
  Pokhara   |   400  |  250  |
  Butwal    |   200  |  100  |
  ----------+--------+-------|
  (Q2, Q3, Q4 have similar tables)
```

**Note:** This 3D cube is the basis for all OLAP operations below.

### 1. Roll-Up (Aggregation / Climbing the Hierarchy):

**Roll-Up:** Roll-Up moves UP the dimension hierarchy - from a more detailed level to a more summarized level. It reduces the number of dimensions or aggregates data along a dimension hierarchy. Equivalent to adding a GROUP BY clause in SQL.

**Roll-Up Operation**

```
  BEFORE Roll-Up (detailed - by Quarter and City):
  Location  | Q1   | Q2   | Q3   | Q4   |
  ----------+------+------+------+------+
  Kathmandu | 800  | 820  | 780  | 900  |  (Laptop + Phone combined)
  Pokhara   | 650  | 660  | 620  | 700  |
```

```
Butwal     | 300  | 290  | 310  | 340  |

Roll-Up TIME: Quarter -> Year (sum all quarters)
(also Roll-Up LOCATION: City -> Province)

AFTER Roll-Up (summarized - by Year and Province):
Province    | Year 2023 |
-----------+-----------|
Bagmati     |   3300    |  (Kathmandu summed across all quarters)
Gandaki     |   2630    |  (Pokhara summed across all quarters)
Lumbini     |   1240    |  (Butwal summed across all quarters)
-----------+-----------|
TOTAL       |   7170    |  <- Grand total (maximum roll-up)
```

**Note:** Roll-Up = aggregation. Moving from Quarter to Year sums all 4 quarters. Moving from City to Province sums all cities in that province.

## 2. Drill-Down (Disaggregation / Descending the Hierarchy):

**Drill-Down:** Drill-Down is the REVERSE of Roll-Up. It moves DOWN the hierarchy - from a summarized level to more detailed data. It reveals finer-grained data that was aggregated in the higher level.

**Drill-Down Operation**

```
BEFORE Drill-Down (by Year):
Product  | Year 2023 |
---------+-----------|
Laptop   |   4200    |
Phone    |   2970    |

Drill-Down TIME: Year -> Quarter

AFTER Drill-Down (by Quarter):
Product  |  Q1  |  Q2  |  Q3  |  Q4  |
---------+------+------+------+------+
Laptop   |  900 |  940 |  920 | 1140 |  (now see each quarter)
Phone    |  650 |  670 |  640 |  810 |

Can drill further: Quarter -> Month -> Week -> Day
```

**Note:** Drill-Down reveals detail hidden in aggregated data. Used when an anomaly is spotted in roll-up data: "Why was Q4 so much higher?" -> drill down to see monthly/product breakdown.

### 3. Slice (Fix One Dimension):

**Slice:** Slice selects a SINGLE VALUE for ONE DIMENSION, reducing the n-dimensional cube to an (n-1)-dimensional subcube. Like cutting a single slice from a loaf of bread.

**Slice Operation**

```
3D Cube: Time x Location x Product

Slice Time = Q1  (fix time dimension to Q1)
Result: 2D table showing Location x Product for Q1 only

BEFORE Slice: 3D cube (Time x Location x Product)

AFTER Slice (Time = Q1):  2D slice
Location  | Laptop | Phone |
----------+--------+-------|
Kathmandu |   500  |  300  |  <- only Q1 data visible
Pokhara   |   400  |  250  |
Butwal    |   200  |  100  |

SQL equivalent:
SELECT location, product, SUM(sales)
FROM FactSales WHERE time_quarter = 'Q1'
GROUP BY location, product;
```

**Note:** Slice = WHERE clause on one dimension. Reduces 3D cube to 2D table.

### 4. Dice (Fix Multiple Dimensions):

**Dice:** Dice selects a SUBCUBE by specifying MULTIPLE values (ranges) for TWO OR MORE dimensions simultaneously. Like cutting a smaller cube from the larger data cube.

**Dice Operation**

```
Dice: Location IN (Kathmandu, Pokhara)
      Time IN (Q1, Q2)
      Product = Phone

Result: sub-cube with only specified values

AFTER Dice:
Location  |  Q1  |  Q2  |  (Product=Phone only)
----------+------+------+
```

```
Kathmandu |  300 |  320 |
Pokhara   |  250 |  230 |

SQL equivalent:
SELECT location, time_quarter, SUM(sales)
FROM FactSales
WHERE location IN ('Kathmandu','Pokhara')
  AND time_quarter IN ('Q1','Q2')
  AND product = 'Phone'
GROUP BY location, time_quarter;
```

**Note:** Dice = WHERE clause on multiple dimensions. Produces a smaller subcube.

## 5. Pivot (Rotate / Transpose):

**Pivot:** Pivot (also called Rotate) ROTATES the data cube to view it from a different perspective - swapping the axes of a 2D view. Like rotating a physical cube to see a different face. Does not change the data, only the presentation.

**Pivot Operation**

```
BEFORE Pivot (Time on columns, Location on rows):
Location  |  Q1  |  Q2  |  Q3  |  Q4  |
----------+------+------+------+------+
Kathmandu |  800 |  820 |  780 |  900 |
Pokhara   |  650 |  660 |  620 |  700 |
Butwal    |  300 |  290 |  310 |  340 |

PIVOT (swap: Location on columns, Time on rows)

AFTER Pivot (Location on columns, Time on rows):
Time |  Kathmandu |  Pokhara |  Butwal |
-----+------------+----------+---------+
Q1   |    800     |    650   |   300   |
Q2   |    820     |    660   |   290   |
Q3   |    780     |    620   |   310   |
Q4   |    900     |    700   |   340   |
```

**Note:** Pivot = same data viewed from different angle. Useful when you want to see cities as columns rather than rows, or products as rows rather than columns.

## Summary of OLAP Operations:

| Operation | Description | Direction | SQL Analogy |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Roll-Up | Aggregate along a dimension hierarchy. | Bottom-up (detail -> summary) | GROUP BY higher level |
| Drill-Down | Disaggregate to more detail. | Top-down (summary -> detail) | GROUP BY lower level |
| Slice | Fix one dimension to a single value. | Reduces cube by 1 dimension | WHERE dim = value |
| Dice | Select subcube with multiple dimension conditions. | Subset of existing cube | WHERE dim1 IN (...) AND dim2 IN (...) |
| Pivot | Rotate the viewing angle of the cube. | No data change, only view changes | TRANSPOSE / column swap |

## SECTION 4: Data Mining - Concepts and Classification

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh*

**Q8.** What is Data Mining? How does it differ from traditional database queries? What are the major tasks of Data Mining? Explain the KDD (Knowledge Discovery in Databases) process.

**[2+2+2+2]**
*2081 Chaitra 2079 Chaitra 2075 Bhadra*

**Answer:** Data Mining - Concepts and KDD Process

**Data Mining:** Data Mining is the process of discovering useful PATTERNS, CORRELATIONS, ANOMALIES, and KNOWLEDGE from large databases and data warehouses. It uses techniques from statistics, machine learning, and database systems to extract non-obvious, previously unknown, and potentially useful information hidden in massive datasets. Also called Knowledge Discovery in Databases (KDD).

### Traditional Database Query vs Data Mining:

| Feature | Traditional DB Query | Data Mining |
|---|---|---|
| Nature of Question | KNOWN question seeking a specific answer. | UNKNOWN patterns to be DISCOVERED from data. |
| Example | "How many laptops were sold in Q1 2023?" | Discover: "Which products are frequently bought together?" |
| User Involvement | User specifies exactly what data to retrieve. | Algorithm discovers patterns without explicit question. |
| Output | Specific answer to a specific query. | Patterns, rules, models, clusters - new knowledge. |

| | | |
|---|---|---|
| Technique | SQL SELECT with WHERE, GROUP BY, etc. | Machine learning algorithms: decision trees, clustering, Apriori, etc. |
| Scale | Works on small to medium datasets efficiently. | Designed for very large datasets (GB to TB). |
| Hypothesis | User has a hypothesis and tests it. | No prior hypothesis - exploratory discovery. |

## Major Tasks of Data Mining:

| Task | Description | Algorithm Examples | Application |
|---|---|---|---|
| Classification | Assign items to predefined categories/classes. Learn a model from labeled training data, then classify new data. | Decision Tree, Naive Bayes, SVM, k-NN, Random Forest | Email spam/not-spam. Patient disease diagnosis. Credit risk assessment. |
| Clustering | Group similar items together without predefined classes. Unsupervised learning. | K-Means, DBSCAN, Hierarchical Clustering | Customer segmentation. Document grouping. Gene expression analysis. |
| Association Rule Mining | Discover rules showing which items co-occur frequently. "If A then B." | Apriori Algorithm, FP-Growth | Market basket analysis. Web navigation patterns. Medical diagnosis rules. |
| Regression | Predict a CONTINUOUS numeric value based on input features. | Linear Regression, Polynomial Regression | Predict house prices. Forecast sales. Stock price prediction. |
| Anomaly Detection | Find unusual patterns that deviate significantly from the norm. | Statistical methods, Isolation Forest | Credit card fraud detection. Network intrusion detection. |
| Summarization | Create compact descriptions of data subsets. | Generalization, Aggregation | Data cube summary reports. Descriptive statistics. |

## KDD (Knowledge Discovery in Databases) Process:

| KDD Process - Steps from Raw Data to Knowledge |
|---|
| |
| `Raw Data (databases, files, web logs, etc.)` |
| `\|` |
| `Step 1: DATA SELECTION` |
| `Choose relevant data subsets for mining.` |
| `Target Data` |

```
        |
  Step 2: DATA PREPROCESSING / CLEANING
        Handle missing values, remove noise, resolve inconsistencies.
        Preprocessed Data
        |
  Step 3: DATA TRANSFORMATION
        Normalize, aggregate, encode, feature selection/engineering.
        Transformed Data
        |
  Step 4: DATA MINING
        Apply mining algorithms: classification, clustering,
        association rules, regression, etc.
        Patterns / Models
        |
  Step 5: PATTERN EVALUATION / INTERPRETATION
        Evaluate interestingness of discovered patterns.
        Filter trivial/obvious patterns. Validate with domain experts.
        Interesting Patterns
        |
  Step 6: KNOWLEDGE REPRESENTATION
        Visualize and present knowledge: decision trees, rule sets,
        charts, reports, dashboards.
        Knowledge (Actionable Insights)
```

**Note:** Data Mining is just ONE step (Step 4) in the entire KDD process. The majority of effort goes into data preprocessing (Steps 2-3), which is sometimes called "data wrangling."

**Q9.** What is Classification in Data Mining? Explain the Decision Tree algorithm with a worked example. Draw a Decision Tree for a given dataset. What are its advantages and limitations?

**[2+6+2]**

*2082 Kartik 2081*
*Ashwin 2079 Chaitra*

**Answer:** Classification and Decision Tree Algorithm

**Classification:** Classification is a supervised data mining task that learns a MODEL (classifier) from a LABELED training dataset, then uses the model to PREDICT the CLASS LABEL of new, unseen instances. "Supervised" means the training data has known correct labels.

**Decision Tree:**

**Decision Tree:** A Decision Tree is a tree-structured classification model where: INTERNAL NODES represent tests on attributes (features). BRANCHES represent test outcomes (attribute values). LEAF NODES represent class labels (final classification). To classify a new instance: start at root, answer each attribute test, follow the branch, reach a leaf with the class prediction.

**Decision Tree - Worked Example (Will a Person Buy a Laptop?):**

```
Training Dataset:

| Age Group | Income   | Student? | Credit | Buys_Laptop? |
|-----------|---------|----------|--------|--------------|
| Young     | High    | No       | Fair   | No           |
| Young     | High    | No       | Exlnt  | No           |
| Middle    | High    | No       | Fair   | Yes          |
| Senior    | Medium  | No       | Fair   | Yes          |
| Senior    | Low     | Yes      | Fair   | Yes          |
| Senior    | Low     | Yes      | Exlnt  | No           |
| Middle    | Low     | Yes      | Exlnt  | Yes          |
| Young     | Medium  | No       | Fair   | No           |
| Young     | Low     | Yes      | Fair   | Yes          |
| Senior    | Medium  | Yes      | Fair   | Yes          |
| Young     | Medium  | Yes      | Exlnt  | Yes          |
| Middle    | Medium  | No       | Exlnt  | Yes          |
| Middle    | High    | Yes      | Fair   | Yes          |
| Senior    | Medium  | No       | Exlnt  | No           |
```

**Decision Tree Built from Above Dataset**

```
                    [Age Group?]
                   /     |      \
              Young    Middle   Senior
               /         |         \
         [Student?]    [YES]    [Credit?]
          /      \               /     \
         No      Yes          Fair    Excellent
         |        |             |          |
       [NO]    [YES]         [YES]       [NO]

Classification Rules extracted from the tree:
  IF Age=Young AND Student=No  THEN Buys_Laptop=No
  IF Age=Young AND Student=Yes THEN Buys_Laptop=Yes
  IF Age=Middle                THEN Buys_Laptop=Yes
  IF Age=Senior AND Credit=Fair      THEN Buys_Laptop=Yes
  IF Age=Senior AND Credit=Excellent THEN Buys_Laptop=No

Classify new instance: Age=Young, Income=Low, Student=Yes, Credit=Fair
  -> Age=Young -> Student? -> Yes -> Prediction: Buys_Laptop = YES
```

**Note:** Decision tree selects the BEST attribute to split at each node using information-theoretic measures. The goal is to maximize homogeneity (purity) in child nodes after each split.

## How Decision Tree Selects the Best Split Attribute:

The most common measure used to select the splitting attribute is INFORMATION GAIN (used in ID3 algorithm) or GINI INDEX (used in CART algorithm).

| Measure | Formula Concept | Goal |
|---|---|---|
| Entropy H(S) | H(S) = -SUM(p_i * log2(p_i)). Measures impurity/disorder of a set S. | Low entropy = pure class (all same label). High entropy = mixed classes. |
| Information Gain | Gain(S,A) = H(S) - weighted average of H(child nodes after split on A). | SELECT attribute A with HIGHEST information gain. Maximizes purity gain. |
| Gini Index | Gini(S) = 1 - SUM(p_i^2). Smaller = purer. | SELECT attribute with LOWEST Gini index after split. |

**Advantages and Limitations of Decision Trees:**

| Advantages | Limitations |
|---|---|
| Easy to understand and visualize. Rules are human-readable. | Can OVERFIT training data - very deep trees memorize noise. |
| No need for data normalization or scaling. | Unstable: small changes in data can produce very different trees. |
| Handles both categorical and numerical attributes. | Biased toward attributes with more levels (information gain favors high cardinality). |
| Fast to classify new instances (follow path from root to leaf). | Poor performance when classes are imbalanced. |
| Can handle missing values with strategies (most common value, etc.). | Single tree: high variance. Solution: Random Forest (ensemble of trees). |

**Q10.** What is the Naive Bayes Classifier? Explain the principle of Naive Bayes classification with an example. What does "Naive" mean in Naive Bayes?

**[2+4+2]**
*2081 Ashwin 2078 Chaitra 2074 Bhadra*

**Answer:** Naive Bayes Classifier

**Naive Bayes Classifier:** Naive Bayes is a probabilistic classification algorithm based on BAYES' THEOREM with the "NAIVE" assumption that all features (attributes) are CONDITIONALLY INDEPENDENT of each other given the class label. Despite this oversimplified assumption (which is rarely true), Naive Bayes works remarkably well in practice, especially for text classification and spam filtering.

## Bayes' Theorem:

**Bayes Theorem and Naive Bayes Formula**

```
  Bayes Theorem:
  P(C|X) = P(X|C) * P(C) / P(X)

  Where:
  P(C|X) = Posterior probability: P(class C given features X) <- WHAT WE
WANT
  P(X|C) = Likelihood: P(features X given class C)
  P(C)   = Prior probability: P(class C) in training data
  P(X)   = Evidence: P(features X) - same for all classes, so we ignore it

  Naive Bayes Classifier:
  C* = argmax_C [ P(C) * PRODUCT(P(x_i|C)) for each feature x_i ]
     = argmax_C [ P(C) * P(x_1|C) * P(x_2|C) * ... * P(x_n|C) ]

  "Naive" assumption: features are INDEPENDENT given the class.
  P(X|C) = P(x_1|C) * P(x_2|C) * ... * P(x_n|C)
  (product of individual feature probabilities)
```

**Note:** The "Naive" independence assumption simplifies computation enormously: instead of computing the joint probability of all features together, compute individual feature probabilities and multiply them.

## Naive Bayes Example - Email Spam Classification:

```
Training Data (emails labeled as Spam/Not-Spam):

| Email  | Contains "free"? | Contains "win"? | Contains "dear"? | Label
|
|--------|-----------------|-----------------|------------------|----------
-|
| E1     |    Yes          |    Yes          |    No            | Spam
|
| E2     |    Yes          |    No           |    No            | Spam
|
| E3     |    No           |    Yes          |    No            | Spam
|
| E4     |    No           |    No           |    Yes           | Not Spam
|
| E5     |    No           |    No           |    Yes           | Not Spam
|
| E6     |    Yes          |    No           |    No            | Not Spam
|

Counts:
  Total Spam = 3,  Total Not-Spam = 3
  P(Spam) = 3/6 = 0.5      P(Not-Spam) = 3/6 = 0.5
```

```
  P(free=Yes | Spam) = 2/3    P(free=Yes | Not-Spam) = 1/3
  P(win=Yes  | Spam) = 2/3    P(win=Yes  | Not-Spam) = 0/3 = 0
  P(dear=Yes | Spam) = 0/3 = 0  P(dear=Yes | Not-Spam) = 2/3

  [Laplace smoothing to avoid zero probabilities:
   P(win=Yes|Not-Spam) = (0+1)/(3+2) = 0.2  (add 1 to count, add #classes
to denominator)]

Classify new email: free=Yes, win=Yes, dear=No

  P(Spam | email) proportional to:
  P(Spam) * P(free=Yes|Spam) * P(win=Yes|Spam) * P(dear=No|Spam)
  = 0.5 * (2/3) * (2/3) * (3/3) = 0.5 * 0.667 * 0.667 * 1.0 = 0.222

  P(Not-Spam | email) proportional to:
  P(Not-Spam) * P(free=Yes|Not-Spam) * P(win=Yes|Not-Spam) * P(dear=No|Not-
Spam)
  = 0.5 * (1/3) * (0.2) * (1/3) = 0.5 * 0.333 * 0.2 * 0.333 = 0.011

  P(Spam) >> P(Not-Spam) -> Classified as: SPAM
```

## SECTION 5: Clustering and Association Rule Mining

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh*

**Q11.** What is Clustering in Data Mining? Explain the K-Means Clustering algorithm with a step-by-step example using coordinates. What are the stopping criteria? Compare K-Means with hierarchical clustering.

**[2+6+2]**

*2082 Kartik 2081 Chaitra 2079 Chaitra*

**Answer:** Clustering and K-Means Algorithm

**Clustering:** Clustering is an UNSUPERVISED data mining task that groups a set of objects into CLUSTERS such that objects in the same cluster are MORE SIMILAR to each other than to objects in other clusters. There are NO predefined labels - the algorithm discovers natural groupings in the data.

**K-Means Clustering Algorithm:**

**K-Means:** K-Means partitions n data points into K clusters by iteratively assigning each point to the nearest cluster centroid and updating the centroids. K (number of clusters) must be specified by the user beforehand.

## K-Means Algorithm Steps:

* Step 1: Choose K (number of clusters). Initialize K centroids (randomly or using K-Means++ strategy).
* Step 2: ASSIGNMENT STEP - Assign each data point to the NEAREST centroid (using Euclidean distance).
* Step 3: UPDATE STEP - Recalculate each centroid as the MEAN (average) of all points assigned to it.
* Step 4: Repeat Steps 2-3 until CONVERGENCE: no point changes cluster assignment (or max iterations reached).

## K-Means Worked Example (K=2, 2D points):

```
Data Points: A(1,1), B(1.5,2), C(3,4), D(5,7), E(3.5,5), F(4.5,5)
K = 2 clusters

STEP 1: Initialize centroids randomly
  C1 = A(1,1)    C2 = D(5,7)

ITERATION 1:
  Assignment - compute distance from each point to C1 and C2:
  dist(A, C1) = sqrt((1-1)^2+(1-1)^2) = 0.0   dist(A, C2) = 7.2  -> assign
to C1
  dist(B, C1) = sqrt((1.5-1)^2+(2-1)^2) = 1.12  dist(B, C2) = 6.1 -> assign
to C1
  dist(C, C1) = sqrt((3-1)^2+(4-1)^2) = 3.61   dist(C, C2) = 3.61 -> tie,
assign C1
  dist(D, C1) = sqrt((5-1)^2+(7-1)^2) = 7.2    dist(D, C2) = 0.0  -> assign
to C2
  dist(E, C1) = sqrt((3.5-1)^2+(5-1)^2) = 4.72  dist(E, C2) = 2.5 -> assign
to C2
  dist(F, C1) = sqrt((4.5-1)^2+(5-1)^2) = 5.32  dist(F, C2) = 2.1 -> assign
to C2

  Cluster 1: {A, B, C}    Cluster 2: {D, E, F}

  Update centroids:
  New C1 = mean of {A(1,1), B(1.5,2), C(3,4)} = ((1+1.5+3)/3, (1+2+4)/3) =
(1.83, 2.33)
  New C2 = mean of {D(5,7), E(3.5,5), F(4.5,5)} = ((5+3.5+4.5)/3,(7+5+5)/3)
= (4.33, 5.67)

ITERATION 2: Re-assign with new centroids
  Check if any point changes cluster... (most remain same)
  C: dist(C, new C1)=2.29, dist(C, new C2)=2.07 -> C moves to Cluster 2!

  Cluster 1: {A, B}       Cluster 2: {C, D, E, F}

  Update centroids:
  New C1 = ((1+1.5)/2, (1+2)/2) = (1.25, 1.5)
  New C2 = ((3+5+3.5+4.5)/4, (4+7+5+5)/4) = (4.0, 5.25)
```

```
ITERATION 3: Re-assign...
  No point changes cluster. CONVERGED!

Final Clusters:
  Cluster 1 (Centroid 1.25, 1.5): A(1,1), B(1.5,2)  <- Low value group
  Cluster 2 (Centroid 4.0, 5.25): C(3,4), D(5,7), E(3.5,5), F(4.5,5) <-
High value group
```

**K-Means Stopping Criteria:**

* No data point changes its cluster assignment (convergence).
* Centroids do not move (or move less than a threshold epsilon).
* Maximum number of iterations reached (prevents infinite loop).

**K-Means Advantages and Limitations:**

| Advantages | Limitations |
|---|---|
| Simple and easy to implement and understand. | Must specify K in advance (difficult without domain knowledge). |
| Scales well to large datasets (O(n) per iteration). | Sensitive to initial centroid positions - may converge to local minimum. |
| Fast convergence in practice. | Only works well with spherical (globular) clusters of similar size. |
| Works well when clusters are compact and well-separated. | Sensitive to outliers - outliers can significantly shift centroids. |

**Q12.** What is Association Rule Mining? Define support, confidence, and lift. Explain the Apriori Algorithm with a complete step-by-step example. How is the minimum support threshold used?

**[2+2+6]**
*2082 Kartik 2079 Chaitra 2077 Chaitra*

**Answer:** Association Rule Mining and Apriori Algorithm

**Association Rule Mining:** Association rule mining discovers INTERESTING RELATIONSHIPS (associations) between variables in large databases. It finds rules of the form: {A, B} -> {C} meaning "if a transaction contains A and B, it likely also contains C." Most famous application: MARKET BASKET ANALYSIS - "Customers who buy diapers also buy beer."

**Key Measures for Association Rules:**

| Measure | Formula | Meaning | Example: {Bread} -> {Butter} |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Support | support(A->B) = count(A U B) / total_transactions | Fraction of transactions containing BOTH A and B. Measures frequency. | Out of 100 transactions, 30 have both bread AND butter. Support = 30/100 = 0.30 (30%). |
| Confidence | conf(A->B) = support(A U B) / support(A) | Among all transactions with A, fraction that ALSO contain B. Measures strength. | Of 40 transactions with bread, 30 also have butter. Confidence = 30/40 = 0.75 (75%). |
| Lift | lift(A->B) = confidence(A->B) / support(B) | How much MORE likely is B given A, compared to buying B randomly. >1=positive correlation. | support(Butter)=0.50. Lift = 0.75/0.50 = 1.5. Buying bread makes buying butter 1.5x more likely. |

## Apriori Algorithm:

**Apriori Algorithm:** The Apriori algorithm discovers frequent itemsets (item combinations that appear in many transactions) and generates association rules from them. It uses the APRIORI PROPERTY (anti-monotone property): Any SUBSET of a frequent itemset MUST ALSO be frequent. This "prune" principle dramatically reduces the search space by eliminating candidate itemsets whose subsets are already infrequent.

## Apriori Algorithm Steps:

* Step 1: Find all FREQUENT 1-ITEMSETS (single items with support >= min_support). Call this L1.
* Step 2: Generate CANDIDATE 2-ITEMSETS (C2) from L1 by combining frequent 1-itemsets.
* Step 3: Prune C2: remove any candidate whose subset is NOT in L1 (Apriori property).
* Step 4: Scan database to count support of remaining candidates. Keep those with support >= min_support -> L2.
* Step 5: Repeat: generate Ck from L(k-1), prune, count support, get Lk.
* Step 6: STOP when no new frequent itemsets are found.
* Step 7: Generate association rules from all frequent itemsets: for each frequent set F, try all rules A->B where A union B = F, A and B non-empty. Keep rules where confidence >= min_confidence.

## Apriori Worked Example:

```
Transaction Database (5 transactions):
  T1: {Bread, Milk, Butter}
  T2: {Bread, Butter}
  T3: {Milk, Butter, Eggs}
  T4: {Bread, Milk, Eggs}
  T5: {Bread, Milk, Butter, Eggs}

minimum support (min_sup) = 3/5 = 60%
minimum confidence (min_conf) = 70%

=== PASS 1: Find Frequent 1-Itemsets (L1) ===

  Item     | Count | Support | Frequent?
  --------+-------+---------+-----------
```

```
  Bread   |   4   |  4/5=80%|   YES
  Milk    |   4   |  4/5=80%|   YES
  Butter  |   4   |  4/5=80%|   YES
  Eggs    |   3   |  3/5=60%|   YES

  L1 = {Bread}, {Milk}, {Butter}, {Eggs}

=== PASS 2: Generate C2 from L1 (all pairs) ===

  C2 = {Bread,Milk}, {Bread,Butter}, {Bread,Eggs},
       {Milk,Butter}, {Milk,Eggs}, {Butter,Eggs}

  Pruning: All subsets of each pair are in L1. No pruning needed.

  Count support for each pair:
  {Bread,Milk}   : T1,T4,T5 = 3 -> 60% YES
  {Bread,Butter} : T1,T2,T5 = 3 -> 60% YES
  {Bread,Eggs}   : T4,T5    = 2 -> 40% NO  <- BELOW min_sup, remove
  {Milk,Butter}  : T1,T3,T5 = 3 -> 60% YES
  {Milk,Eggs}    : T3,T4,T5 = 3 -> 60% YES
  {Butter,Eggs}  : T3,T5    = 2 -> 40% NO  <- BELOW min_sup, remove

  L2 = {Bread,Milk}, {Bread,Butter}, {Milk,Butter}, {Milk,Eggs}

=== PASS 3: Generate C3 from L2 ===

  Candidate: {Bread,Milk,Butter} (join {Bread,Milk} and {Bread,Butter})
  Pruning check: subsets {Bread,Milk}YES, {Bread,Butter}YES,
{Milk,Butter}YES -> KEEP

  Count: {Bread,Milk,Butter}: T1,T5 = 2 -> 40% < 60% -> NOT frequent

  No frequent 3-itemsets found. STOP.

=== GENERATE ASSOCIATION RULES from L2 ===

From {Bread, Milk} (support=60%):
  Bread -> Milk: conf = 3/4 = 75% >= 70% -> RULE ACCEPTED
  Milk  -> Bread: conf = 3/4 = 75% >= 70% -> RULE ACCEPTED

From {Milk, Eggs} (support=60%):
  Milk -> Eggs:  conf = 3/4 = 75% >= 70% -> RULE ACCEPTED
  Eggs -> Milk:  conf = 3/3 = 100% >= 70% -> RULE ACCEPTED

Final Strong Association Rules (support>=60%, confidence>=70%):
  Bread -> Milk      (sup=60%, conf=75%)
  Milk  -> Bread     (sup=60%, conf=75%)
```

```
Milk  -> Eggs      (sup=60%, conf=75%)
Eggs  -> Milk      (sup=60%, conf=100%)
```

**Most Repeated:** "Explain Apriori algorithm with an example showing all passes, candidate generation, pruning, and rule generation." This is one of the most commonly asked data mining questions. Show all passes clearly with the itemset frequency table.

## SECTION 6: Data Mining Applications and Summary

*Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin*

**Q13.** What are the applications of Data Mining in various fields? Explain how data mining is used in: banking, healthcare, retail, and telecommunications. What are ethical concerns in data mining?

**[2+4+2]**
*2081 Ashwin 2079 Chaitra 2075 Bhadra*

**Answer:** Applications of Data Mining

| Field | Data Mining Application | Specific Example |
|---|---|---|
| Banking & Finance | Fraud detection, credit risk assessment, customer segmentation, stock market prediction. | Classify transactions as fraud/not-fraud using decision trees. Cluster customers by spending patterns for targeted credit card offers. |
| Healthcare / Medical | Disease diagnosis, drug discovery, treatment recommendation, epidemic prediction. | Classify patient symptoms to predict disease (cancer detection from scan). Mine hospital records to find which treatments work best for specific conditions. |
| Retail / E-commerce | Market basket analysis, recommendation systems, customer churn prediction, inventory management. | Amazon: "Customers who bought X also bought Y" (association rules). Predict which customers will stop buying (churn prediction using classification). |
| Telecommunications | Network fault detection, call detail record analysis, customer retention, spam call detection. | Cluster call patterns to detect abnormal usage (fraud). Predict customer churn before contract end. |
| Education (EDM) | Student performance prediction, learning path recommendation, dropout prediction. | Predict which students are at risk of failing using past performance data. Recommend personalized learning resources. |

| | | |
|---|---|---|
| Web/Social Media | Click-stream analysis, social network analysis, sentiment analysis, targeted advertising. | Cluster web browsing patterns for targeted ads. Mine Twitter data for public sentiment on a topic. |
| Government / Security | Tax fraud detection, crime prediction, security threat analysis. | IRS uses data mining to identify tax fraud patterns. Police use crime pattern analysis for preventive deployment. |
| Bioinformatics | Gene expression analysis, protein function prediction, drug-gene interaction discovery. | Cluster gene expression data to identify related genes. Mine literature for drug interaction patterns. |

**Ethical Concerns in Data Mining:**

| Concern | Description |
|---|---|
| Privacy Violation | Mining personal data (location, health, behavior) without proper consent. Individuals may not know their data is being mined or what patterns are being extracted about them. |
| Data Security | Large data warehouses are high-value targets for hackers. A breach can expose sensitive information about millions of individuals. |
| Discrimination and Bias | If training data reflects historical biases (racial, gender, socioeconomic), the mined patterns and classification models will perpetuate and amplify those biases. |
| Informed Consent | Data collected for one purpose (e.g., customer transactions) may be mined for another purpose (health inference, political profiling) without the user's knowledge or consent. |
| Data Ownership | Who owns the patterns discovered from data? The data owner? The mining company? Legal frameworks are still evolving. |
| Misuse of Results | Association rules and patterns can be misused: e.g., retailers manipulating pricing based on inferred personal circumstances. |

**Q14.** Write SQL queries to compute: (a) ROLLUP for total sales by region and product, (b) find frequent 2-itemsets using self-join in SQL, (c) use window functions to rank customers by total purchase per city.

**[2+4+2]**
*2081 Ashwin 2078*
*Chaitra 2076 Baisakh*

**Answer:** SQL for Data Warehouse and Mining Tasks

**(a) ROLLUP for Sales Subtotals and Grand Total:**

```
-- ROLLUP: Get sales totals at region+product, region, and grand total
levels
SELECT
    COALESCE(region, 'ALL REGIONS') AS region,
```

```
    COALESCE(product, 'ALL PRODUCTS') AS product,
    SUM(sales_amount) AS total_sales,
    COUNT(*) AS num_transactions
FROM FactSales
    JOIN DimLocation ON FactSales.location_id = DimLocation.location_id
    JOIN DimProduct  ON FactSales.product_id  = DimProduct.product_id
GROUP BY ROLLUP(region, product)
ORDER BY region, product;

-- Result rows:
-- Bagmati  | Laptop        | 1200000 | 45   <- specific region+product
-- Bagmati  | Phone         |  800000 | 60
-- Bagmati  | ALL PRODUCTS  | 2000000 | 105  <- region subtotal
-- Gandaki  | Laptop        |  900000 | 35
-- ...
-- ALL REGIONS | ALL PRODUCTS | 5500000 | 300 <- grand total
```

**(b) Find Frequent 2-Itemsets Using SQL Self-Join (Market Basket Analysis):**

```
-- Transactions table: (trans_id, item)
-- Find all item PAIRS that appear together in >= 30% of transactions

SELECT
    t1.item AS item1,
    t2.item AS item2,
    COUNT(*) AS co_occurrence,
    COUNT(*) * 1.0 / (SELECT COUNT(DISTINCT trans_id) FROM Transactions) AS
support
FROM Transactions t1
JOIN Transactions t2
    ON t1.trans_id = t2.trans_id   -- same transaction
    AND t1.item < t2.item          -- avoid duplicates: (A,B) not (B,A)
GROUP BY t1.item, t2.item
HAVING support >= 0.30             -- minimum support threshold 30%
ORDER BY support DESC;

-- Result: frequent 2-itemsets with their support values
-- item1  | item2  | co_occurrence | support
-- Bread  | Milk   |     30        |  0.60
-- Milk   | Eggs   |     30        |  0.60
-- Bread  | Butter |     30        |  0.60
```

**(c) Window Function: Rank Customers by Total Purchase per City:**

```
-- Rank customers by their total purchases within each city
SELECT
    customer_name,
    city,
    total_purchase,
```

```
    RANK() OVER (
        PARTITION BY city          -- restart ranking for each city
        ORDER BY total_purchase DESC -- rank by highest purchase first
    ) AS city_rank,
    DENSE_RANK() OVER (
        PARTITION BY city
        ORDER BY total_purchase DESC
    ) AS dense_city_rank
FROM (
    SELECT
        c.customer_name,
        c.city,
        SUM(s.sales_amount) AS total_purchase
    FROM FactSales s
    JOIN DimCustomer c ON s.customer_id = c.customer_id
    GROUP BY c.customer_name, c.city
) customer_totals
ORDER BY city, city_rank;

-- Result:
-- customer_name | city      | total_purchase | city_rank
-- Ram Sharma    | Kathmandu |   150000       |    1
-- Sita Thapa    | Kathmandu |   130000       |    2
-- Hari Poudel   | Kathmandu |   130000       |    2  <- same rank (tie)
-- Gita Sharma   | Kathmandu |   110000       |    4  <- rank skips 3
-- ...           | Pokhara   |   ...          |    1  <- restarts in
Pokhara
```

## QUICK REVISION: Chapter 9 - Key Points Summary

| Topic | Key Points | Expected Marks |
|---|---|---|
| Data Warehouse | Subject-oriented + Integrated + Non-volatile + Time-variant (Inmon's 4 characteristics). Read-only historical data for analysis. CANNOT use OLTP for analytics: performance conflict, no history, fragmented data, normalized schema too complex. | 2-4 |
| DW Architecture | Tier1: Data Sources (OLTP, ERP, files). Tier2: ETL (Extract-Transform-Load) staging. Tier3: EDW + Data Marts + OLAP + Reporting tools. Data Mart = department-specific DW subset. | 2-4 |
| OLTP vs OLAP | OLTP: operations, writes, normalized, current, fast (ms), clerks. OLAP: analysis, reads, denormalized, historical, slow (sec-min), analysts. OLAP uses DW as data source. | 3-6 |

| | | |
|---|---|---|
| Data Cube | Multidimensional array indexed by dimensions (Time, Location, Product). Measures = numeric facts (sales, qty). Dimension hierarchy: Day->Month->Quarter->Year. Cell = intersection of dimension values. | 2-4 |
| Star Schema | Fact table (FKs + measures) at center, denormalized dimension tables around it. Simple 1-level JOINs. Fastest for OLAP. Most popular. Snowflake: normalized dimensions (more JOINs). Fact Constellation: multiple fact tables sharing dimensions. | 4-8 |
| ETL | Extract (read from sources) -> Transform (clean, standardize, integrate: fix nulls/encoding/duplicates/dates) -> Load (append to DW). Most effort in Transform. Incremental loads after initial full load. | 2-4 |
| OLAP Operations | Roll-Up: aggregate up hierarchy (city->province). Drill-Down: disaggregate down. Slice: fix 1 dimension (WHERE). Dice: fix multiple dimensions (subcube). Pivot: rotate viewing angle. All equivalent to SQL GROUP BY + WHERE variants. | 4-8 |
| Data Mining & KDD | KDD: Select -> Preprocess -> Transform -> Mine -> Evaluate -> Represent. Data mining = finding non-obvious patterns. Tasks: Classification (supervised), Clustering (unsupervised), Association Rules, Regression, Anomaly Detection. | 2-4 |
| Decision Tree | Internal node=attribute test, branch=outcome, leaf=class. Select best split by Information Gain (ID3) or Gini Index (CART). Rules: IF conditions THEN class. Pros: interpretable. Cons: overfitting, instability. | 3-6 |
| Naive Bayes | P(C\|X) proportional to P(C) * PRODUCT(P(xi\|C)). Assumes feature independence ("naive"). Bayes theorem. Laplace smoothing for zero probabilities. Fast, works well for text/spam. | 2-4 |
| K-Means Clustering | 1) Init K centroids. 2) Assign each point to nearest centroid (Euclidean dist). 3) Recompute centroids as mean of cluster. 4) Repeat until no change. K must be given. Sensitive to initialization and outliers. | 3-6 |
| Apriori Algorithm | support(A->B) = count(AUB)/total. confidence(A->B) = support(AUB)/support(A). lift(A->B) = conf/support(B). Apriori property: all subsets of frequent itemset are frequent. Passes: L1->C2->prune->L2->C3->... Stop when no new frequent sets. Generate rules where confidence >= min_conf. | 4-8 |
| DW Applications | Banking: fraud detection. Healthcare: disease diagnosis. Retail: market basket, recommendation. Telecom: churn prediction. Ethical issues: privacy, bias, consent, data ownership. | 1-3 |